

GridPulse: An End-to-End Analytical Data Pipeline for Australia’s National Electricity Market

Aditya Moon Pranjal Desai

School of Computer Science, The University of Sydney
Sydney, Australia
adityamoon690@gmail.com

Abstract

Modern electricity grids emit a continuous stream of high-frequency telemetry that is difficult to convert into trustworthy, decision-ready insight. This paper presents *GridPulse*, an end-to-end, reproducible data-engineering pipeline over Australia’s National Electricity Market (NEM). GridPulse ingests five-minute facility-level power and carbon-dioxide-equivalent (CO₂e) emissions data from the OpenElectricity v4 REST API, cleans, aggregates and geocodes it, enforces a nine-check quality gate, loads a normalised analytical schema into an embedded DuckDB warehouse, and models it into star-shaped, tested marts using dbt. The entire lineage is expressed as a Dagster software-defined asset graph and served through two complementary interfaces: a real-time MQTT-to-Dash map and an extended Streamlit analytics dashboard. Over a verified one-week window the pipeline processes 668,134 facility-interval records across 541 facilities, producing ≈ 3.62 TWh of energy and ≈ 2.15 Mt CO₂e. The principal finding is a pronounced decoupling of energy from carbon: fossil fuels supply roughly 68% of energy but nearly 100% of emissions, implying that decarbonisation is dominated by a small number of coal units. We describe the architecture, the design rationale at each stage, and the analytical results, and we argue that pairing a hot operational view with a warm analytical view over one tested model keeps the two surfaces consistent by construction.

Index Terms

data engineering, ELT, DuckDB, dbt, Dagster, orchestration, data quality, electricity markets, emissions analytics, data visualisation

I. INTRODUCTION

Australia’s **National Electricity Market (NEM)** is one of the world’s longest interconnected power systems, coordinating supply and demand across five regions—New South Wales, Queensland, Victoria, South Australia and Tasmania (Australian Energy Market Operator [AEMO], 2024). Every five minutes hundreds of generating facilities report their output, and this telemetry underpins both market dispatch and the national conversation about decarbonisation. Converting that firehose of data into a form a human can reason about—joining it to facility meta-data, emissions and geography, then serving it interactively—is a canonical data-engineering problem (Kleppmann, 2017).

GridPulse began as Assignment 2 of **COMP5339 (Data Engineering)** at the University of Sydney (University of Sydney, 2026) and was subsequently extended into a portfolio-grade system. The original work established a functioning publisher/subscriber streaming demonstration and a normalised DuckDB schema; the extended project wraps that foundation in the engineering practices that distinguish a demonstration from a maintainable platform: an installable package, a declarative transformation layer, automated data tests, orchestration and reproducible offline replays.

This paper documents the system end to end. Section III presents the architecture; Section IV describes the methodology stage by stage with explicit design rationale; Section V details the data model; Section VI describes the two serving surfaces; Section VII reports analytical results; and Sections VIII–XI cover quality assurance, use

TABLE I: Data sources and their role in the pipeline.

Source	Content	Grain / Volume	Role
OpenElectricity v4 API	Power & CO ₂ e per facility	5-min × facility (668,134)	System of record
Facilities catalogue	Fuel, capacity, coords, region	542 facilities	Dimension
NEM market series	Spot price & demand	5-min (2,016)	Market context
NGER report	Greenhouse/energy figures	Station-year	Census map
ABS region tables	Statistical geography	Region-level	Enrichment

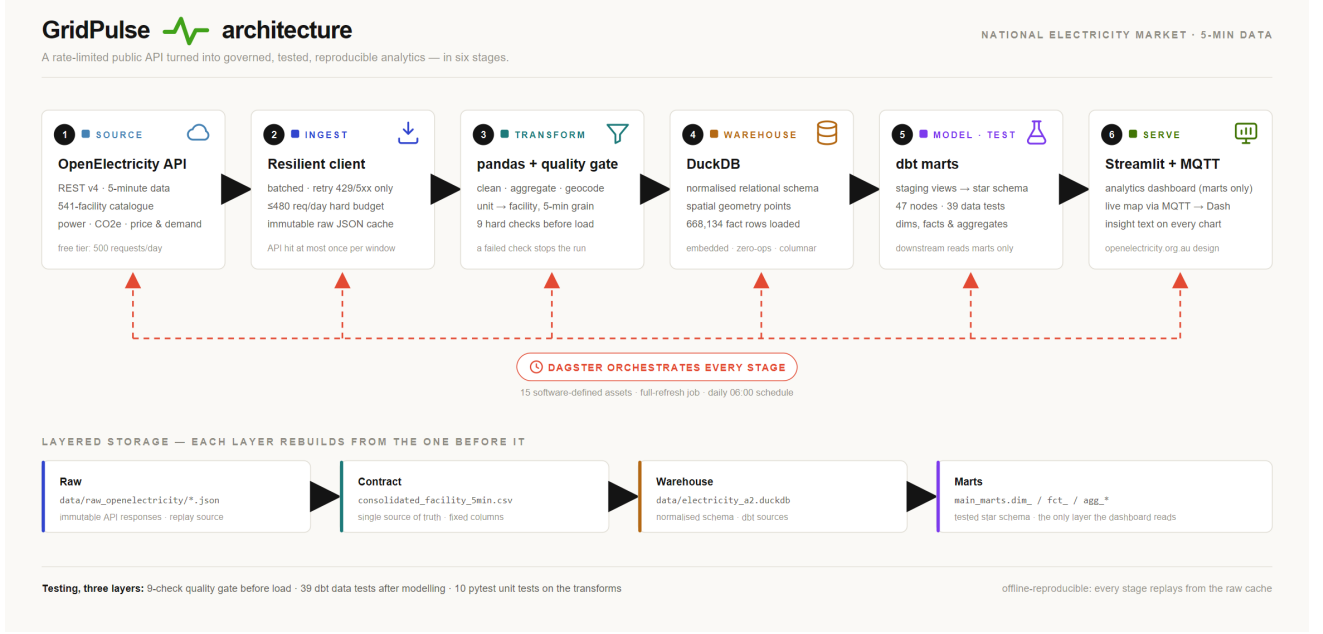


Fig. 1: GridPulse reference architecture. Six stages (source → ingest → transform → warehouse → model → serve) joined by the data flow, with a Dagster control bus orchestrating every stage and a layered-storage strip (raw JSON → contract CSV → DuckDB → dbt marts).

cases, limitations and conclusions. A recurring principle is **layered storage with a single, tested source of truth**: the external API is contacted at most once per window, after which every downstream artefact is reproducible from an immutable cache.

II. DATA SOURCES

GridPulse draws on one primary streaming source and several reference datasets, all openly licensed. Table I summarises them. Generation and emissions data are published by **OpenElectricity**, an initiative of The Superpower Institute that exposes NEM telemetry through a modern REST API (OpenElectricity, 2024); facility-level emissions are reported directly, avoiding the need to model fuel-specific emission factors. The National Greenhouse and Energy Reporting (NGER) figures used in a secondary all-Australia map originate from the Clean Energy Regulator’s reporting scheme (Clean Energy Regulator, 2024).

III. SYSTEM ARCHITECTURE

The architecture follows a **layered, medallion-style storage** discipline: data flows through six clearly bounded stages, and each stage writes an immutable or rebuildable artefact so that failures are contained and reruns are cheap (Kleppmann, 2017). Critically, only the first stage contacts the network; everything downstream replays from local artefacts. Figure 1 shows the reference architecture.

The **layered-storage contract** is the backbone of the design. Immutable raw JSON is the evidentiary record of what the API returned; the consolidated CSV is the single contract that every replay reads; the DuckDB schema is the queryable warehouse; and the dbt marts are the tested, analysis-ready layer the dashboards consume. Because the dashboards read *only* the marts, the presentation layer can never diverge from the tested model.

IV. PIPELINE METHODOLOGY

Each stage below is described in the same shape—*what* it does, *why* it matters, and *how* it is implemented—so the design intent is legible, not merely the mechanics.

A. Ingestion and API hygiene

What. A typed `OpenElectricityClient` fetches power and emissions for the requested window in ≤ 7 -day batches (the API caps a five-minute request at eight days) and caches every response as raw JSON. **Why.** APIs are rate-limited and occasionally flaky; naive polling burns the request budget and corrupts runs. **How.** The client retries only on transient failures (HTTP 429 and 5xx), honours the `Retry-After` header, tracks a conservative ≤ 480 -requests/day budget, and fails fast on 4xx errors. Each response is written to `data/raw_openelectricity/`, so an interrupted run resumes from cache (OpenElectricity, 2024).

B. Transformation and consolidation

What. Raw long-format JSON is parsed, trimmed to the UTC window, summed from generating *units* to *facilities*, standardised across metrics, and pivoted into a wide analysis-ready table. **Why.** Facilities—not units—are the analytical grain, and downstream code should never see vendor-specific metric names or partial intervals. **How.** `transform.py` produces `consolidated_facility_5min.csv`, the data contract, with one row per facility per five-minute interval and explicit `power_mw`, `energy_mwh` ($= \text{power}/12$) and `emissions_tco2e` columns, built with pandas (McKinney, 2010). Negative power is deliberately retained (batteries charging); emissions are asserted non-negative.

C. Geocoding and spatial enrichment

What. Every facility is resolved to valid coordinates and an Australian state. **Why.** The dashboards are map-first; a missing or out-of-bounds coordinate would drop a station from view. **How.** `geocode.py` validates each latitude/longitude against an Australian bounding box, back-fills the $\approx 2\%$ of facilities lacking API coordinates from NEM region centroids, and derives state. In the warehouse these become DuckDB `GEOMETRY` points via `ST_Point`, with a graceful fallback when the spatial extension is unavailable.

D. Quality gate (pre-load)

What. A dependency-free “expectations” gate runs nine checks before any data is loaded. **Why.** It is far cheaper to reject bad data at the door than to debug a polluted warehouse; a hard fail here stops the run. **How.** `quality.py` asserts row counts, non-null keys, coordinate ranges, metric domains (including the emissions ≥ 0 invariant) and referential coverage. All nine checks pass on the sample week.

E. Warehouse load

What. The validated contract is loaded into a normalised DuckDB schema. **Why.** DuckDB is an embedded, columnar analytical engine—zero-ops, fast on aggregates, and file-based, so the whole warehouse ships in the repository (Raasveldt & Mühleisen, 2019). **How.** `load.py` populates `FACILITY`, `FUEL_TYPE`, `FACILITY_POWER_EMISSIONS` and `MARKET_PRICE_DEMAND` as the warehouse source layer.

F. Dimensional modelling with dbt

What. dbt transforms the raw warehouse into staging views and then star-shaped marts. **Why.** A declarative, version-controlled, *tested* transformation layer makes the model auditable and the business logic explicit (dbt Labs, 2024). **How.** Staging models (`stg_facilities`, `stg_power_emissions`, `stg_market`) type and scope the data—notably filtering to the five NEM regions, which drops one stray Western-Australian facility (hence 541 modelled vs. 542 catalogued). Marts include `dim_facility`, the `fact_facility_interval` fact (grain: facility $\times$ 5-min interval) and three aggregate marts. A full dbt build runs **47 nodes with 39 data tests**, all passing.

TABLE II: Principal marts in `main_marts`.

Mart	Type	Grain / Purpose
<code>dim_facility</code>	Dimension	1 row/facility (541); attributes, fuel, capacity, location
<code>fct_facility_interval</code>	Fact	facility×5-min (668,134); power, energy, emissions, intensity
<code>agg_fuel_mix_daily</code>	Aggregate	date×fuel; energy & emissions
<code>agg_region_intensity</code>	Aggregate	region; energy, intensity, renewable share
<code>agg_diurnal_profile</code>	Aggregate	hour×category; the average day

G. Orchestration with Dagster

What. The whole lineage—`ingest` → `warehouse` → `dbt marts`—is expressed as software-defined assets in a single Dagster graph with a daily 06:00 refresh schedule. **Why.** Asset-centric orchestration provides one lineage view, per-run metadata and re-materialisation of only what changed, rather than opaque task scripts (Dagster Labs, 2024). **How.** A `DagsterDbtTranslator` maps warehouse tables to dbt sources so the dbt models appear natively in the asset graph; the definitions load **15 assets** with a full-refresh job and schedule.

V. DATA MODEL AND WAREHOUSE SCHEMA

The warehouse is organised into three DuckDB schemas: `main` (raw sources), `main_staging` (typed dbt views) and `main_marts` (tested tables). The marts follow a classic star/aggregate pattern so that common questions—“what did each fuel produce?”, “which region is dirtiest?”, “what does the average day look like?”—are answered by reading a single small table. Table II lists the principal marts.

The fact table carries both UTC (`interval_ts`) and local display time (`interval_local`, Australia/Sydney), plus derived `energy_mwh` and `intensity_tco2e_mwh` so intensive metrics never have to be recomputed by consumers.

VI. SERVING LAYER: TWO COMPLEMENTARY DASHBOARDS

A deliberate design choice is that **the same warehouse feeds two distinct serving surfaces**, each optimised for a different question. They are not duplicates: one answers “*what is happening right now?*” and the other “*what does the whole window tell us?*”. This split of a hot, real-time plane from a warm, analytical plane mirrors production data systems (Kleppmann, 2017).

A. Live-streaming dashboard (real-time)

Artefact: `Assignment2_Dashboard_Group156.ipynb` (MQTT → Plotly Dash). This is the **live-data dashboard**: the subscriber half of a publisher/subscriber system. A companion notebook streams facility records over the public MQTT broker (`broker.hivemq.com`, topic `comp5339/group156/electricity`) using the lightweight publish/subscribe MQTT protocol designed for telemetry (OASIS, 2019). The subscriber runs an MQTT client on a background thread, maintains thread-safe rolling state, and a Dash app polls that state every 1.5 seconds to redraw a live map (Plotly Technologies Inc., 2015). Each facility appears at its true location, coloured and sized by a user-toggled metric (power or CO₂e); region and fuel filters, a click-through station pop-up, a live generation-mix stack, an emissions-intensity trace and a rolling price/demand panel update as messages arrive. As an integration touch, every received message is matched by `facility_code` and written back into the shared `FACILITY_POWER_EMISSIONS` table—closing the loop between the stream and the warehouse.

B. Extended analytics dashboard (historical)

Artefact: `dashboard/app_streamlit.py` (Streamlit + Plotly, reads the dbt marts only). This is the **extended-features dashboard**, built with Streamlit (Streamlit Inc., 2024) and modelled on the visual language of *openelectricity.org.au*. It reads exclusively from the tested `main_marts` tables, so nothing on screen can drift from the validated model. Where the live dashboard shows the latest instant, this one lets an analyst explore the entire window across four sections: *Overview* (KPIs, a week-long stacked generation profile—Fig. 2—a fuel summary table, energy-vs-emissions donuts, and demand & price); *Facilities* (a filterable table beside a map of all 541 facilities with click-through detail, plus an all-Australia NGER census layer); *Analysis* (regional small-multiples, the diurnal duck curve, leaderboards, intensity by fuel); and *About* (an embedded architecture case study). Table III contrasts the two surfaces.

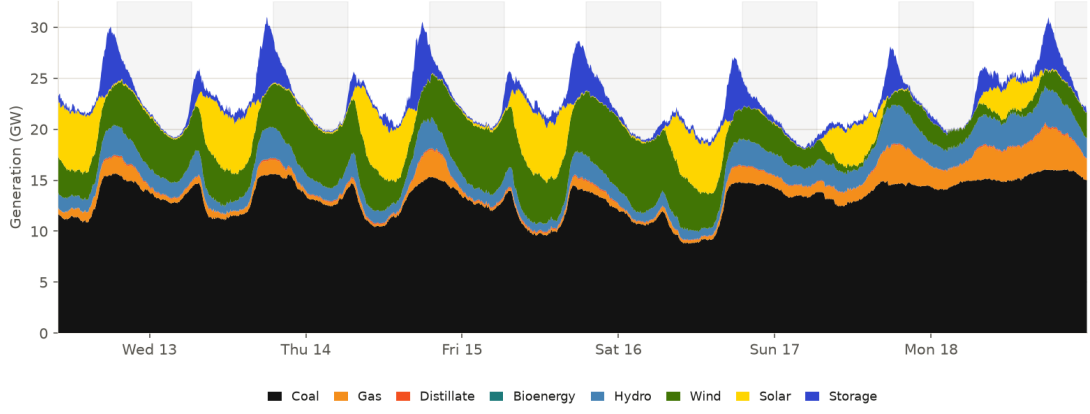


Fig. 2: The extended dashboard’s signature Overview chart, reproduced from the marts: week-long generation (GW) stacked by fuel group, with evening night-bands. Coal forms a flat baseload floor; solar bulges through each midday; wind and storage fill the peaks—the same view the Streamlit app renders interactively.

TABLE III: The two dashboards compared.

	Live (Dash)	Extended (Streamlit)
Data path	MQTT stream; writes to DuckDB	Reads dbt marts only
Temporality	Real-time, latest instant	Whole window, historical
Question	“What is happening now?”	“What does the window tell us?”
Refresh	1.5 s polling	Cached queries (10-min TTL)
Best for	Operational monitoring	Analysis, storytelling

VII. RESULTS AND INSIGHTS

All figures below are computed directly from the tested marts—the same tables the dashboards read—so the paper and the product agree by construction. Values are for the verified week 12–18 May 2026.

A. The energy–carbon decoupling

The single most important result is that **energy and carbon are decoupled**. Coal supplies the majority of energy but virtually all of the emissions, while renewables contribute nearly a third of energy at almost zero carbon. Figure 3 shows this as two near-mirror-image distributions, and Fig. 4 as concentric energy/emissions rings; Table IV gives the underlying values.

B. Regional intensity

Where energy is made matters as much as how much. Figure 5 ranks the five regions by energy, carbon intensity and renewable share, with values in Table V. **Victoria** is the dirtiest grid at **0.767 tCO₂e/MWh** on brown coal, while hydro-rich **Tasmania** is roughly **60× cleaner** at 0.013. New South Wales is the largest producer at 1.20 TWh.

C. The average day: a “duck curve”

Folding the week into a single average day exposes the classic solar “duck curve”. Figure 6 shows solar swelling through the middle of the day (roughly 09:00–15:00 local), displacing gas, while coal holds a flat baseload floor and storage fills the evening peak.

D. Market rhythm: demand and price

NEM-wide operational demand follows a strong daily cycle (averaging ≈ 21.3 GW, peaking near 28 GW), and the volume-weighted spot price rides the evening ramp, spiking well above its $\approx \text{A\$}93/\text{MWh}$ average when demand tightens. Figure 7 plots both on deliberately separate axes.

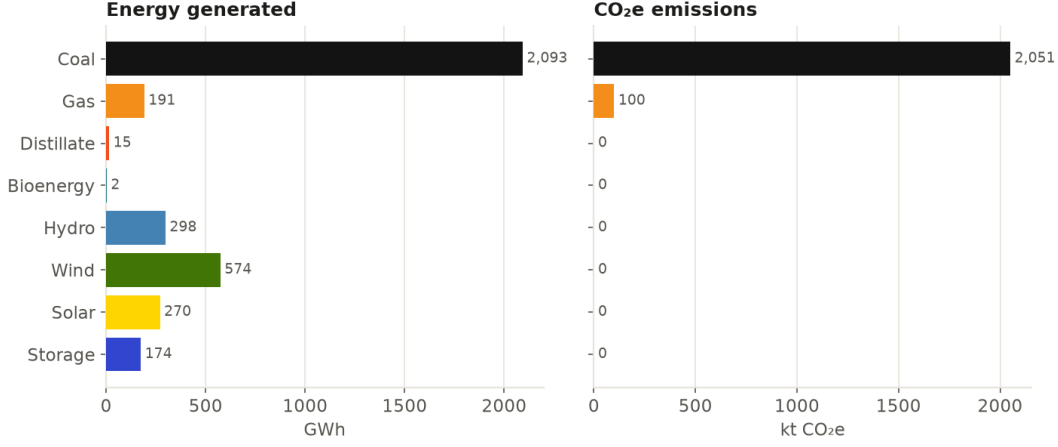


Fig. 3: Energy generated (left) versus CO₂e emissions (right) by fuel group. Renewables generate substantial energy with effectively zero emissions; brown and black coal dominate the carbon side.

TABLE IV: Energy and emissions by fuel technology (sample week).

Fuel technology	GWh	kt CO ₂ e	Class
Black coal	1,509.2	1,354.2	Fossil
Brown coal	583.6	696.3	Fossil
Wind	573.9	0.0	Renewable
Hydro	297.8	0.0	Renewable
Solar (utility)	270.3	0.0	Renewable
Battery / storage	173.5	0.0	Storage
Gas (all types)	191.5	99.5	Fossil
Distillate	15.1	0.0	Fossil
Bioenergy	2.5	0.1	Renewable

E. Facility leaderboard

Table VI lists the ten largest generators by energy; all are coal. Because emissions are so concentrated, the marts make the decarbonisation lever explicit: retiring or abating a handful of these units moves the national number far more than uniform, grid-wide efficiency gains.

VIII. DATA QUALITY AND TESTING

Trustworthiness is a first-class feature, enforced by three independent test layers that run at different points in the lifecycle (Table VII). Data is validated *before* it enters the warehouse and asserted again *after* it is modelled, and the Python code paths are covered by unit tests. Two invariants deserve emphasis. First, **negative power is valid** and retained, because batteries legitimately draw power while charging; a naive “clip to zero” would silently destroy information. Second, **emissions must be non-negative**, asserted both in the pre-load gate and as a dbt range test. Together these encode domain knowledge into the pipeline.

IX. USE CASES

Although built as a portfolio project, GridPulse maps onto genuine analytical needs: *operational monitoring* (the live map gives a control-room view of present power and emissions); *emissions accounting* (facility-level CO₂e rolled up to fuel and region); *decarbonisation planning* (the leaderboard and intensity marts quantify which units carry the carbon); *market analysis* (aligning mix with demand and price exposes the evening ramp and the value of storage); *load-shifting signals* (the diurnal profile shows when the grid is cleanest); and an *engineering reference* for layered storage, quality gating, dbt modelling and Dagster orchestration.

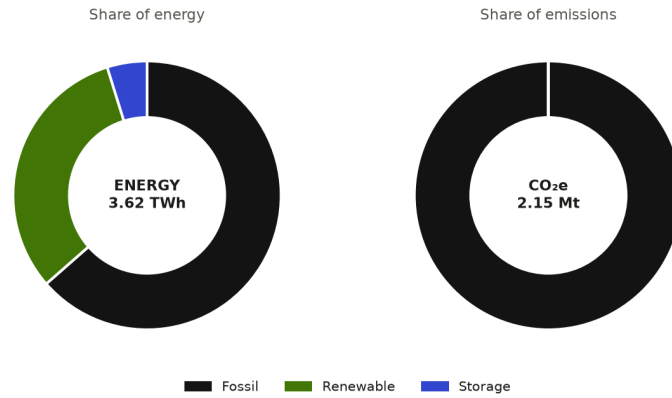


Fig. 4: Share of energy (left ring) and share of emissions (right ring) by category. Fossil fuels are $\approx 68\%$ of energy but $\approx 100\%$ of emissions; storage shifts energy in time without generating any.

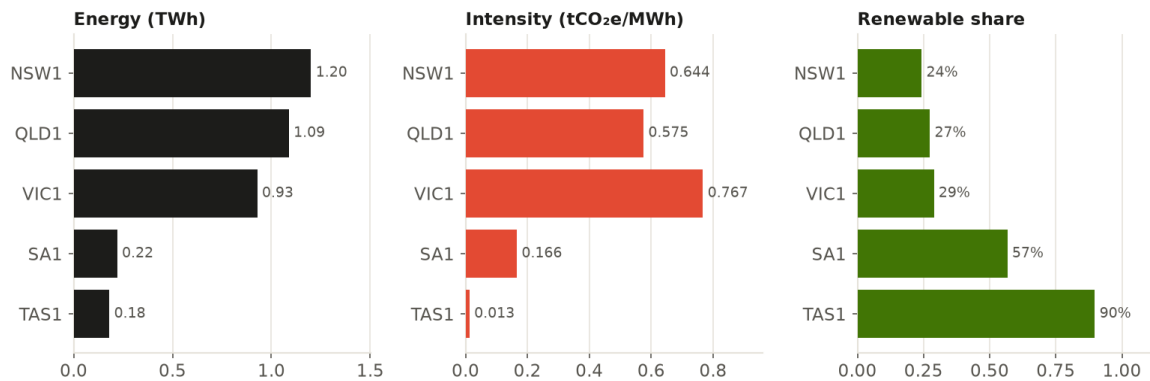


Fig. 5: Regional comparison—energy (TWh), carbon intensity (tCO₂e/MWh) and renewable share by NEM region.

X. LIMITATIONS AND FUTURE WORK

The committed analytical window is a single week, chosen so the repository stays small and reproducible offline; the pipeline already supports multi-week fetches where an API key is supplied. The public MQTT broker used for the live demo is best-effort and unauthenticated, appropriate for demonstration rather than production. Emissions are taken as reported by the API rather than independently modelled. Planned extensions include incremental dbt models with partitioned Parquet for longer histories; a short-horizon price/emissions forecast mart (e.g. Prophet or XGBoost) feeding a “shift your load now?” signal; and continuous integration (GitHub Actions) running `pytest` and `dbt build` on every push, with scheduled cloud Dagster runs.

XI. CONCLUSION

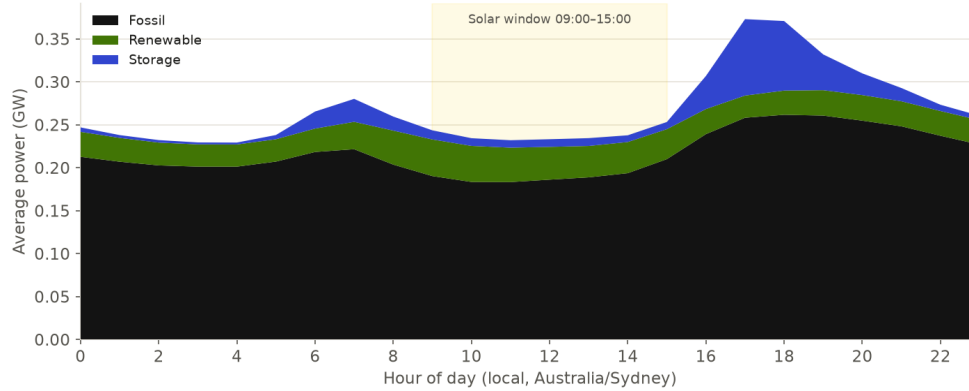
GridPulse takes a noisy, five-minute firehose of electricity-market telemetry and turns it—through a disciplined chain of ingestion, transformation, quality gating, dimensional modelling and orchestration—into a tested warehouse and two complementary dashboards. The analytical payoff is a clear, defensible story: Australia’s grid emissions are concentrated in a handful of coal units while renewables already deliver a third of energy at near-zero carbon. The engineering payoff is a reproducible, testable, orchestrated system that behaves like a small production data platform rather than a one-off script. The two-dashboard design—pairing a real-time MQTT map with a deep Streamlit analytics surface over one shared model—shows how the same trustworthy data can serve both operational and analytical audiences without divergence.

REFERENCES

Australian Energy Market Operator. (2024). *About the National Electricity Market (NEM)*. <https://aemo.com.au/en/energy-systems/electricity/national-electricity-market-nem>

TABLE V: Regional energy, emissions and intensity (sample week).

Region	Fac.	TWh	tCO ₂ e	Int.	Ren.
NSW1	98	1.200	773,306	0.644	24.3%
QLD1	86	1.088	625,890	0.575	27.4%
VIC1	81	0.928	712,105	0.767	28.9%
SA1	60	0.220	36,546	0.166	56.7%
TAS1	28	0.181	2,308	0.013	89.7%

**Fig. 6:** Average generation by hour of day (local time), stacked by fuel category. The shaded band marks the solar window; coal remains a near-constant baseload beneath the renewable swing.

- Clean Energy Regulator. (2024). *National Greenhouse and Energy Reporting scheme*. Australian Government. <https://cer.gov.au/schemes/national-greenhouse-and-energy-reporting-scheme>
- Dagster Labs. (2024). *Dagster documentation: Software-defined assets*. <https://docs.dagster.io/>
- dbt Labs. (2024). *dbt documentation: Building a dbt project*. <https://docs.getdbt.com/docs/build/projects>
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- Kleppmann, M. (2017). *Designing data-intensive applications*. O’Reilly Media.
- McKinney, W. (2010). Data structures for statistical computing in Python. In S. van der Walt & J. Millman (Eds.), *Proceedings of the 9th Python in Science Conference* (pp. 56–61). <https://doi.org/10.25080/Majora-92bf1922-00a>
- OASIS. (2019). *MQTT version 5.0: OASIS standard*. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- OpenElectricity. (2024). *OpenElectricity API (v4)* [Data set]. The Superpower Institute. <https://openelectricity.org.au/>
- Plotly Technologies Inc. (2015). *Collaborative data science*. <https://plot.ly>
- Raasveldt, M., & Mühleisen, H. (2019). DuckDB: An embeddable analytical database. In *Proceedings of the 2019 International Conference on Management of Data* (pp. 1981–1984). ACM. <https://doi.org/10.1145/3299869.3320212>
- Streamlit Inc. (2024). *Streamlit documentation*. <https://docs.streamlit.io/>
- The pandas development team. (2020). *pandas-dev/pandas: Pandas*. Zenodo. <https://doi.org/10.5281/zenodo.3509134>
- University of Sydney. (2026). *COMP5339: Data engineering—Assignment 2 specification*. School of Computer Science.

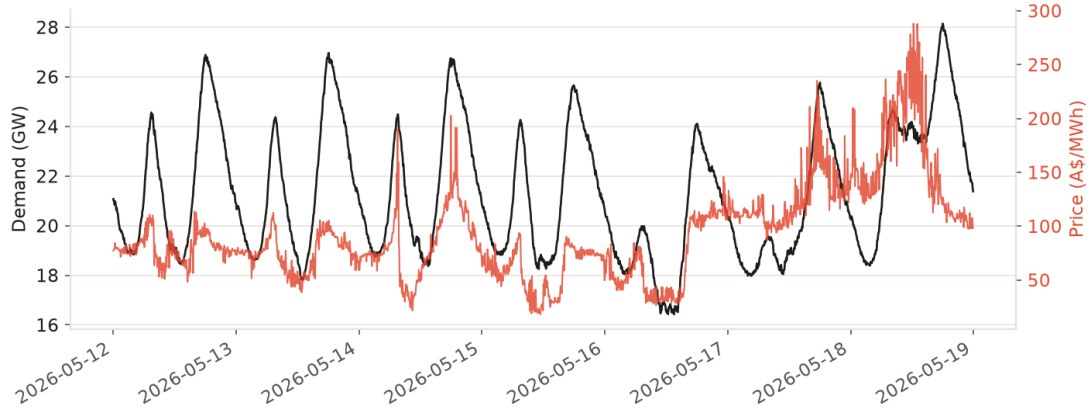


Fig. 7: NEM-wide operational demand (black, left axis) and spot price (red, right axis) at five-minute resolution across the sample week.

TABLE VI: Top ten facilities by energy generated (sample week).

#	Facility	Region	Fuel	GWh
1	Eraring	NSW1	Black coal	298.8
2	Bayswater	NSW1	Black coal	297.3
3	Loy Yang A	VIC1	Brown coal	227.0
4	Yallourn W	VIC1	Brown coal	182.9
5	Loy Yang B	VIC1	Brown coal	173.8
6	Tarong	QLD1	Black coal	168.0
7	Vales Point B	NSW1	Black coal	127.5
8	Stanwell	QLD1	Black coal	125.9
9	Mt Piper	NSW1	Black coal	123.8
10	Gladstone	QLD1	Black coal	103.0

TABLE VII: Test layers and verified status (5 July 2026).

Layer	Tool	Guards	Result
Unit tests	pytest	Transform, quality, geocode	10/10
Pre-load gate	quality.py	Counts, keys, ranges, invariants	9/9
Post-model	dbt	unique, not_null, relationships, ranges	39 tests
Orchestration	Dagster	Asset-graph integrity	15 assets